

Zadanie 01- Natalia Włodyka

Polecenie zadania:

Celem zadania jest rozwiązanie podanego poniżej układu równań. Do jego rozwiązania wykorzystamy metodę Thomasa, zmodyfikowaną tak, aby brała pod uwagę jedynki w rogach.

Zadany układ:

Do rozwiązania mamy podany układ:

$$\begin{bmatrix} 4 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 4 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 4 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 4 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 4 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{bmatrix}$$

Macierz prawostronna jest prawie trójdzielna, symetryczna oraz rzadka, co musimy wziąć pod uwagę przy rozwiązywaniu układu.

Uzasadnienie wyboru algorytmu:

Zastosowana wariacja algorytmu rozwiązywania Thomasa pozwala na rozwiązanie układu ze złożonością $O(n)$. Algorytm ten stosuje się wobec macierzy trójdzielnych, które tak jak macierze rzadkie, mają w macierzy większości zera.

Metoda Thomasa dla danego układu:

W naszej implementacji musimy wziąć pod uwagę jedynki w prawym górnym oraz lewym dolnym rogu macierzy. Przyjmujemy poniższe oznaczenia:

$$A = \begin{bmatrix} b_1 & c_1 & & & a_1 \\ a_2 & b_2 & c_2 & & \\ & a_3 & b_3 & \ddots & \\ & & \ddots & \ddots & c_{n-1} \\ c_n & & & a_n & b_n \end{bmatrix}, x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix}, d = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \\ \vdots \\ d_n \end{bmatrix}$$

W tym celu tworzymy na podstawie podanego układu dwa układy o rozmiarze $(n-1) \times (n-1)$:

$$\begin{array}{ll} b_2 u_2 + c_2 u_3 = d_2 & b_2 v_2 + c_2 v_3 = -a_2 \\ a_3 u_2 + b_3 u_3 + c_3 u_4 = d_3 & a_3 v_2 + b_3 v_3 + c_3 v_4 = 0 \\ a_i u_{i-1} + b_i u_i + c_i u_{i+1} = d_i & a_i u_{i-1} + b_i u_i + c_i u_{i+1} = 0 \quad i = 4, \dots, n-1 \\ \vdots & \vdots \\ a_n u_{n-1} + b_n u_n = d_n & a_n v_{n-1} + b_n v_n = -c_n \end{array}$$

Przyjmujemy, że $u_1 = 0$ i $v_1 = 1$. Rozwiązania będą miały postać:

$$x_i = u_i + x_1 v_i \quad i = 1, 2, \dots, n$$

Element x_1 znajdujemy według podanego wzoru:

$$x_1 = \frac{d_1 - a_1 u_n - c_1 u_2}{b_1 + a_1 v_n + c_1 v_2}$$

Kod w C++:

```
#include <iostream>
void thomas_algorythm(const int size, double x[], const double a[], const
double b[], const double c[]) {
    double pom[size];
    double v[size];
    pom[1] = c[1] / b[1];
    v[1] = -a[1] / b[1];
    x[1] = x[1] / b[1];

    for (int i = 2; i < size - 1; i++) {
        const double m = 1.0 / (b[i] - a[i] * pom[i - 1]);
        pom[i] = c[i] * m;
        v[i] = (0.0f - a[i] * v[i - 1]) * m;
        x[i] = (x[i] - a[i] * x[i - 1]) * m;
    }

    const double m = 1.0 / (b[size - 1] - a[size - 1] * pom[size - 2]);
    pom[size - 1] = c[size - 1] * m;
    v[size - 1] = (-c[0] - a[size - 1] * v[size - 2]) * m;
    x[size - 1] = (x[size - 1] - a[size - 1] * x[size - 2]) * m;

    for (int i = size - 2; i >= 1; i--) {
        v[i] -= pom[i] * v[i + 1];
        x[i] -= pom[i] * x[i + 1];
    }

    x[0] = (x[0] - a[0] * x[size - 1] - c[0] * x[1]) / (b[0] + a[0] *
v[size - 1] + c[0] * v[1]);

    for (int i = 1; i < size; i++){
        x[i] += x[0] * v[i];
    }

    for (int i = 0; i < size; i++){
        std :: cout << x[i] << std:: endl;
    }
}
int main(){
    const double a[7] = {1,1,1,1,1,1,1};
    const double b[7] = {4,4,4,4,4,4,4};
    const double c[7] = {1,1,1,1,1,1,1};
    double x[7] = {1,2,3,4,5,6,7};
    thomas_algorythm(7,x,a,b,c);
}
```

Wyniki:

$$x[0] = -0.2601$$

$$x[1] = 0.4472$$

$$x[2] = 0.4715$$

$$x[3] = 0.6667$$

$$x[4] = 0.8618$$

$$x[5] = 0.8862$$

$$x[6] = 1.5935$$

Wnioski z zadania:

Jak widać z powyższych wyników, zmodyfikowaną metodą Thomasa można uzyskać w czasie $O(n)$ poprawne rozwiązania macierzy takich jak podanej w zadaniu.